



Università degli Studi di Padova
Corso di Laurea in Informatica
A. A. 2025/26



Relazione

Progetto di Tecnologie Web

Studenti	Sofia De Blasi - 2111014 - sofia.deblasi@studenti.unipd.it
	Nicola Simionato - 2113190 - nicola.simionato.5@studenti.unipd.it
	Nenad Radulovic - 2101059 - nenad.radulovic@studenti.unipd.it
	Giacomo Speggiorin - 2116438 - giacomo.speggiorin@studenti.unipd.it
Indirizzo del sito	https://tecweb.studenti.math.unipd.it/sdeblasi/
Utente	Username: user, Password: user

Indice

1	Introduzione	3
1.1	Com'è nata l'idea	3
1.2	Obiettivo di StudentSpace	3
2	Analisi del lavoro svolto	3
2.1	Processo decisionale	3
2.2	Modalità di lavoro	3
2.3	Divisione del lavoro	4
3	Progettazione	4
3.1	Tipologie di utenti	4
3.2	Analisi dei contenuti	4
3.3	Descrizione delle funzionalità	5
3.4	Architettura del sistema	6
3.5	Struttura del sito	7
3.6	Schema di navigazione e metafora della pesca	7
3.7	Pagine di errore	8
4	Sviluppo	8
4.1	Database	8
4.1.1	Analisi dei requisiti e progettazione logica	8
4.1.2	Schema ER	9
4.1.3	Scelta delle chiavi primarie	9
4.1.4	Schema relazionale	10
4.1.5	Normalizzazione	11
4.1.6	Vincoli di integrità	11
4.2	HTML	12
4.2.1	Meta tag <code>title</code> , <code>description</code> e <code>keywords</code>	12
4.2.2	Meta tag <code>viewport</code>	12
4.2.3	<code><aside></code> in <code>esplora.html</code>	12
4.2.4	Struttura di <code>cardTemplate</code>	12
4.2.5	Gestione delle immagini	12
4.2.6	Form e validazione	13
4.2.7	Messaggio informativo nei redirect verso la pagina di accesso	13
4.3	CSS e decisioni di design	13
4.4	PHP	14
5	Accessibilità	14
5.1	Strumenti di verifica	14
5.2	Tipografia e leggibilità	15
5.3	Colori, contrasti e accessibilità mobile	15
6	Validazione, sicurezza e gestione degli errori	16
6.0.1	Errori lato client (JavaScript)	16
6.0.2	Errori lato server (PHP)	17
7	Configurazione <code>.htaccess</code>	17

8	Test	17
8.1	Testing multiplatforma con e senza JavaScript	17
8.2	Ordine di tabulazione e navigazione da tastiera	17
8.3	Analisi delle prestazioni e SEO	18
8.4	Validazione con strumenti automatici	18
8.5	Studio comportamento utenti	18
9	Strumenti utilizzati	18
9.1	Docker	19
10	Guida all'installazione del progetto	19

1 Introduzione

1.1 Com'è nata l'idea

L'idea del progetto nasce dall'esigenza, comune tra gli studenti universitari, di individuare rapidamente eventi, attività e servizi utili nella propria città universitaria. Attualmente tali informazioni risultano frammentate tra diversi canali (pagine social, bacheche fisiche sparse, etc.), rendendo la ricerca poco immediata.

Da questa osservazione è emersa la necessità di una piattaforma unica che:

- centralizzi la pubblicazione di eventi e iniziative;
- permetta agli studenti di consultarli senza monitorare più fonti;
- offra agli organizzatori un punto di raccolta comune.

La riflessione si è estesa anche ad altri servizi rilevanti per la vita universitaria come ripetizioni, affitti e progetti sperimentali. Da qui l'idea di creare uno spazio organizzato, accessibile e pensato per raccogliere tutto ciò che può risultare utile nella quotidianità studentesca.

1.2 Obiettivo di StudentSpace

StudentSpace propone una *bacheca online* dedicata agli studenti universitari.

Gli obiettivi principali della piattaforma sono:

- fornire a studenti e studentesse un accesso rapido, organizzato e completo a contenuti di interesse quotidiano;
- semplificare la comunicazione per gli organizzatori, offrendo loro un canale unico e mirato per raggiungere un pubblico omogeneo e specifico.

2 Analisi del lavoro svolto

2.1 Processo decisionale

Durante lo sviluppo del progetto sono state necessarie numerose decisioni riguardanti design, funzionalità e contenuti. La maggior parte delle scelte è stata discussa in presenza tra tutti i membri del gruppo, ma sono stati adottati anche canali sincroni e asincroni.

Per le comunicazioni asincrone è stata utilizzata la piattaforma di messaggistica **WhatsApp**, mentre **Discord** è stato impiegato come supporto alle riunioni e per la gestione delle comunicazioni sincrone a distanza.

2.2 Modalità di lavoro

Per garantire flessibilità organizzativa, il gruppo ha scelto una modalità di lavoro prevalentemente asincrona.

Le attività sono state distribuite in modo che ciascun componente lavorasse su aspetti differenti, evitando sovrapposizioni. A supporto di questa organizzazione è stato adottato un sistema di versionamento tramite **Git** con hosting su **GitHub**, utile per tracciare modifiche, gestire branch e mantenere un flusso di lavoro ordinato.

2.3 Divisione del lavoro

Considerata la natura accademica del progetto, si è ritenuto opportuno coinvolgere ogni membro in tutte le principali aree di sviluppo, così da garantire una visione complessiva del funzionamento della piattaforma e una partecipazione equilibrata.

Sono state organizzate riunioni periodiche di allineamento, durante le quali veniva aggiornata la lista delle attività da svolgere e venivano assegnati i compiti in modo bilanciato. Per facilitare la pianificazione e il monitoraggio del lavoro sono state utilizzate le *GitHub Issues*, che hanno permesso di tracciare in modo chiaro le attività, le priorità e lo stato di avanzamento.

3 Progettazione

3.1 Tipologie di utenti

Il sistema prevede due tipologie principali di utenti, differenziate per permessi e funzionalità.

- **Utente non registrato:** Può consultare liberamente le categorie e leggere gli annunci.
 - **Azioni e permessi:** accesso alle pagine *index* ed *esplora*; utilizzo di ricerca e filtri; visualizzazione del dettaglio dell’annuncio e del profilo pubblico del proponente; possibilità di avviare registrazione o accesso.
 - **Limitazioni:** impossibilità di pubblicare, modificare o salvare annunci. In caso di tentativi di azioni riservate, il sistema reindirizza alla pagina di accesso con spiegazione di redirect e con collegamento alla registrazione.
- **Utente registrato:** Dispone di tutte le funzionalità dell’utente non registrato, con accesso aggiuntivo alla gestione dei propri contenuti.
 - **Azioni e permessi:** creazione, modifica ed eliminazione dei propri annunci; consultazione del profilo; gestione degli annunci preferiti e pubblicati.
 - **Limitazioni:** impossibilità di modificare annunci altrui; obbligo di rispettare i vincoli di validazione lato client e server.

3.2 Analisi dei contenuti

La piattaforma funge da bacheca universitaria online, organizzata in quattro categorie principali: **Affitti**, **Esperimenti**, **Eventi** e **Ripetizioni**. L’analisi dei contenuti considera in base ai tipi di annunci: i campi comuni e gli attributi specifici per categoria.

Campi generici Ogni annuncio contiene i seguenti campi comuni:

- **Categoria:** selezionata tra le quattro disponibili (**Affitti**, **Esperimenti**, **Eventi** e **Ripetizioni**).
- **Titolo:** breve descrizione sintetica.
- **Data di pubblicazione:** generata automaticamente.
- **Città:** città universitaria di riferimento.
- **Descrizione:** testo libero con i dettagli dell'annuncio.
- **Autore:** utente registrato che ha pubblicato l'annuncio.

Campi specifici per categoria Oltre ai campi generici, ogni categoria prevede tre attributi specifici:

- | | |
|--|---|
| <ul style="list-style-type: none"> • Affitti: <ul style="list-style-type: none"> – Prezzo mensile – Indirizzo – Numero coinquilini • Esperimenti: <ul style="list-style-type: none"> – Laboratorio di riferimento – Durata prevista – Compenso | <ul style="list-style-type: none"> • Eventi: <ul style="list-style-type: none"> – Data evento – Luogo – Costo entrata • Ripetizioni: <ul style="list-style-type: none"> – Materia – Livello (superiori, università, ecc.) – Prezzo orario |
|--|---|

3.3 Descrizione delle funzionalità

Registrazione: permette la creazione di un account tramite inserimento di: Nome, Cognome, Email, Città universitaria, Password

Accesso: consente l'accesso alle funzionalità riservate.

Pubblicazione annunci: gli utenti registrati possono pubblicare annunci tramite un form dedicato. Campi richiesti:

- **Generici:** titolo, descrizione, categoria.
- **Specifici:** mostrati dinamicamente in base alla categoria selezionata.

Modifica, eliminazione e salvataggio degli annunci:

- Gli utenti registrati possono salvare annunci nei preferiti.
- L'autore può modificare o eliminare i propri annunci.

- Gli annunci eliminati vengono rimossi automaticamente anche dalle liste dei preferiti.

Filtraggio e ricerca degli annunci Il sistema permette di applicare filtri per affinare la ricerca:

- **Filtri generici:** categoria, data di pubblicazione.
- **Filtri specifici:** attributi variabili in base alla categoria selezionata.
- **Ricerca testuale:** su titolo, descrizione ed email dell'autore.

In caso di dati non validi, nei form con attributo `data-validate` l'invio della form viene bloccato. In ogni caso le form vengono validate e ove necessario vengono mostrati messaggi di errore (si veda [la sezione sulla validazione](#)).

3.4 Architettura del sistema

Il sistema è progettato per garantire una chiara separazione tra contenuto, presentazione e comportamento, con l'obiettivo di migliorare manutenibilità, riusabilità e accessibilità.

Struttura generale L'architettura del progetto segue una struttura modulare:

- **Root del progetto:** contiene i file PHP principali (`index.php`, `accedi.php`, `pubblica.php`, `profilo.php`, ecc.), i file di configurazione (`.env`, `env.php`, `apache.conf`) e le pagine di errore (`403.php`, `404.php`, `500.php`).
- **/pages:** template HTML puri utilizzati per la composizione dinamica delle pagine.
- **/css:** fogli di stile suddivisi per contesto:
 - `style.css`: stile principale del sito;
 - `mini.css`: stile per dispositivi con larghezza ≤ 800 px;
 - `print.css`: stile dedicato alla stampa.
- **/js:** script JavaScript (es. `script.js`) per validazione lato client e miglioramento dell'usabilità.
- **/assets:** risorse multimediali statiche (icone, loghi, immagini di interfaccia).
- **/img_annunci:** immagini caricate dagli utenti e associate agli annunci.
- **/database:** script SQL per la creazione del database (`studentSpaceDB.sql`).
- **/relazione:** documentazione finale del progetto, comprensiva del file LATEX e delle immagini utilizzate.

Separazione dei livelli La logica applicativa è implementata nei file PHP, responsabili di gestione delle richieste, validazione lato server, interazione con il database e composizione dinamica delle pagine tramite template.

La presentazione è affidata ai file HTML e CSS, mentre il comportamento dinamico lato client è gestito tramite JavaScript.

3.5 Struttura del sito

La struttura del sito è stata analizzata tramite un modello ad albero, utile per valutarne ampiezza e profondità ed evidenziare i percorsi di navigazione principali.

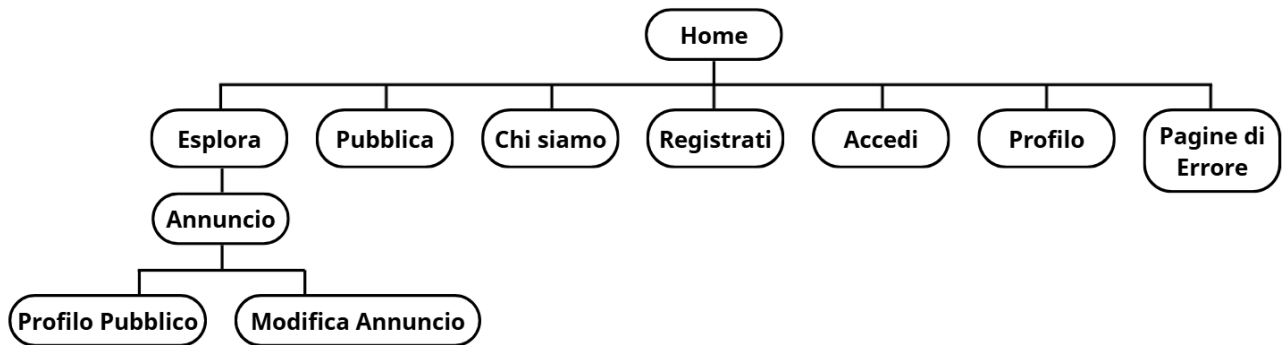


Figura 1: Rappresentazione ad albero della struttura del sito

Dal punto di vista dell'architettura dell'informazione, il sito presenta una **ampiezza (7) maggiore rispetto alla profondità (4)**: le funzionalità principali sono raggiungibili direttamente dalla homepage o con al massimo tre livelli di navigazione. Questa scelta riduce il carico cognitivo e facilita l'orientamento, soprattutto per utenti alla prima visita.

3.6 Schema di navigazione e metafora della pesca

Il sito adotta uno **schema di navigazione ibrido**, basato su una struttura principalmente gerarchica ma arricchita da elementi tipici degli schemi ambigui, che permettono percorsi alternativi di esplorazione.

La navigazione richiama la **metafora della pesca**, grazie alla presenza di strumenti che consentono diversi approcci alla ricerca dei contenuti:

- **Barra di ricerca testuale** → *tiro perfetto*: permette di individuare rapidamente annunci specifici tramite titolo, descrizione o email dell'autore.
- **Filtri generici e specifici** → *trappola per aragoste*: consentono di restringere progressivamente il campo dei risultati.
- **Accesso diretto alle categorie dalla homepage** → *trappola per aragoste / pesca con la rete*: guida verso insiemi omogenei di contenuti.
- **Ultimi annunci inseriti** → *pesca con la rete*: favorisce la scoperta casuale dei contenuti più recenti.
- **Lista dei preferiti e area personale** → *boa di segnalazione*: permette di raccogliere e riorganizzare nel tempo i contenuti di interesse.

3.7 Pagine di errore

Ogni errore è stato progettato per non essere bloccante: sono sempre presenti elementi di interazione chiari, come il pulsante per tornare alla *Home* o il collegamento diretto alla bacheca degli annunci. Questo permette all'utente di recuperare rapidamente la navigazione senza sentirsi "intrappolato" nella pagina di errore. Inoltre, abbiamo scelto di utilizzare un tono leggero e leggermente ironico.

4 Sviluppo

4.1 Database

4.1.1 Analisi dei requisiti e progettazione logica

Entità principali Il modello dati si basa sulle seguenti entità:

- **Utente:** rappresenta gli utenti registrati. `IdUtente` (PK), `Nome`, `Cognome`, `Email` (UNIQUE), `Password`, `Idcittà` (FK)*.
- **Città:** sede universitaria o città di riferimento. `Idcittà` (PK), `Nomecittà` (UNIQUE).
- **Annuncio:** contenuto pubblicato sulla bacheca. `IdAnnuncio` (PK), `Titolo`, `Descrizione`, `DataPubblicazione` (DEFAULT CURRENT_TIMESTAMP), `IdUtente` (FK), `NomeCategoria`, `Idcittà` (FK).
- **Specializzazioni di Annuncio:** tabelle figlie che estendono `Annuncio` con tre attributi specifici:
 - **Affitti:** `PrezzoMensile`, `Indirizzo`, `NumeroInquilini`.
 - **Esperimenti:** `Laboratorio`, `DurataPrevista`, `Compenso`.
 - **Eventi:** `DataEventi`, `Luogo`, `CostoEntrata`.
 - **Ripetizioni:** `Materia`, `Livello`, `PrezzoOrario`.
- **ImmaginiAnnuncio:** immagini associate a un annuncio. `IdImmagine` (PK), `IdAnnuncio` (FK), `Percorso`, `AltText*`, `Decorativa`, `Ordine`.

Relazioni principali

- **Pubblicazione:** `Utente` (1,1) $\rightarrow$ `Annuncio` (0,N).
- **Collocazione:** `Città` (1,1) $\rightarrow$ `Annuncio` (0,N).
- **SedeUniversitaria:** `Città` (1,1) $\rightarrow$ `Utente` (0,N).
- **Associazione immagini:** `Annuncio` (1,1) $\rightarrow$ `ImmaginiAnnuncio` (0,N).
- **Preferiti:** relazione multi-a-multi tra `Utente` e `Annuncio`.

4.1.2 Schema ER

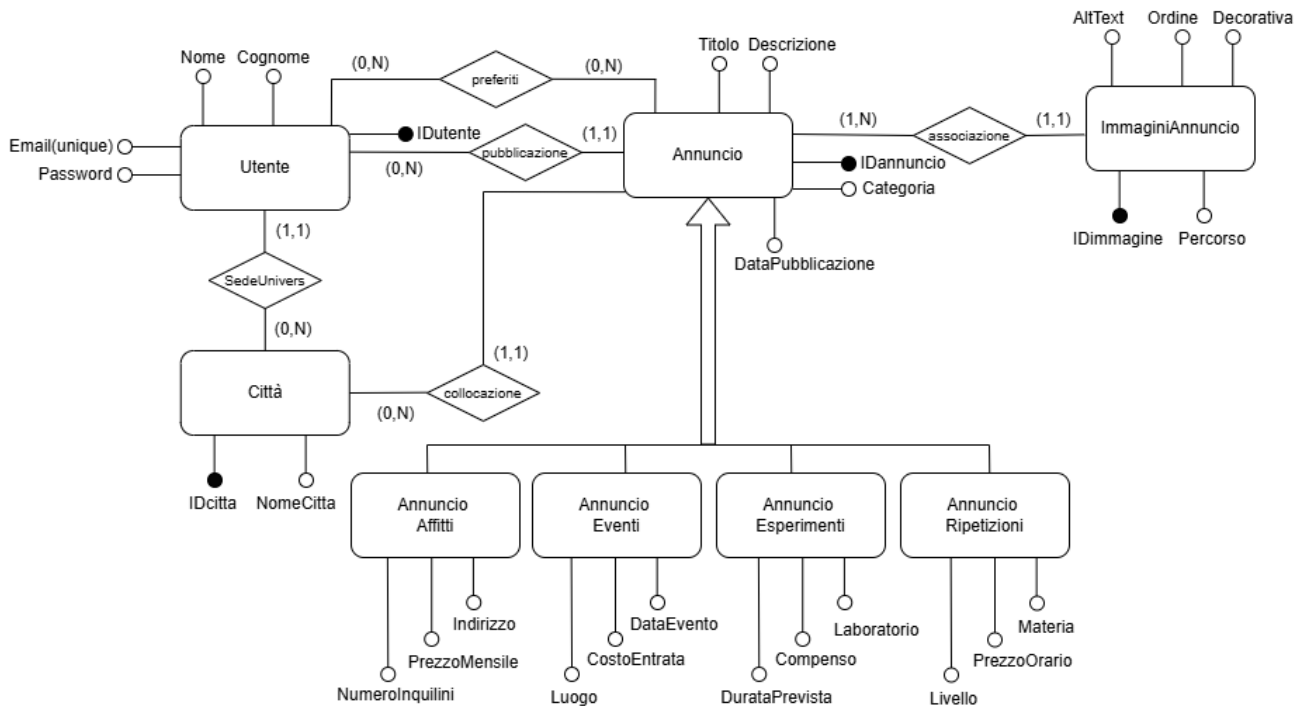


Figura 2: Schema ER

Ristrutturazione della generalizzazione Nel modello concettuale iniziale, **Annuncio** era padre delle quattro categorie specifiche. In fase di progettazione logica la generalizzazione è stata sostituita con associazioni IS-[categoria], scelta che:

- evita la duplicazione dei campi generici;
- mantiene la normalizzazione;
- facilita l'estendibilità del modello;
- separa logicamente i dati specifici per categoria.

È introdotto il vincolo secondo cui ogni annuncio può appartenere ad una sola specializzazione.

Relazione IS-[categoria] **Annuncio** (1,1) → **Specializzazione** (0,1). Ogni annuncio può avere al massimo una specializzazione.

4.1.3 Scelta delle chiavi primarie

Sono stati adottati ID numerici autoincrementali come chiavi primarie per:

- **stabilità**: gli attributi naturali possono cambiare nel tempo;
- **efficienza**: gli ID numerici sono più leggeri negli indici e nelle join;

- **semplicità**: schema più uniforme e leggibile;
- **integrità**: gli attributi naturali restano vincolati con **UNIQUE**;
- **estendibilità**: facilita l'aggiunta di nuove categorie o entità.

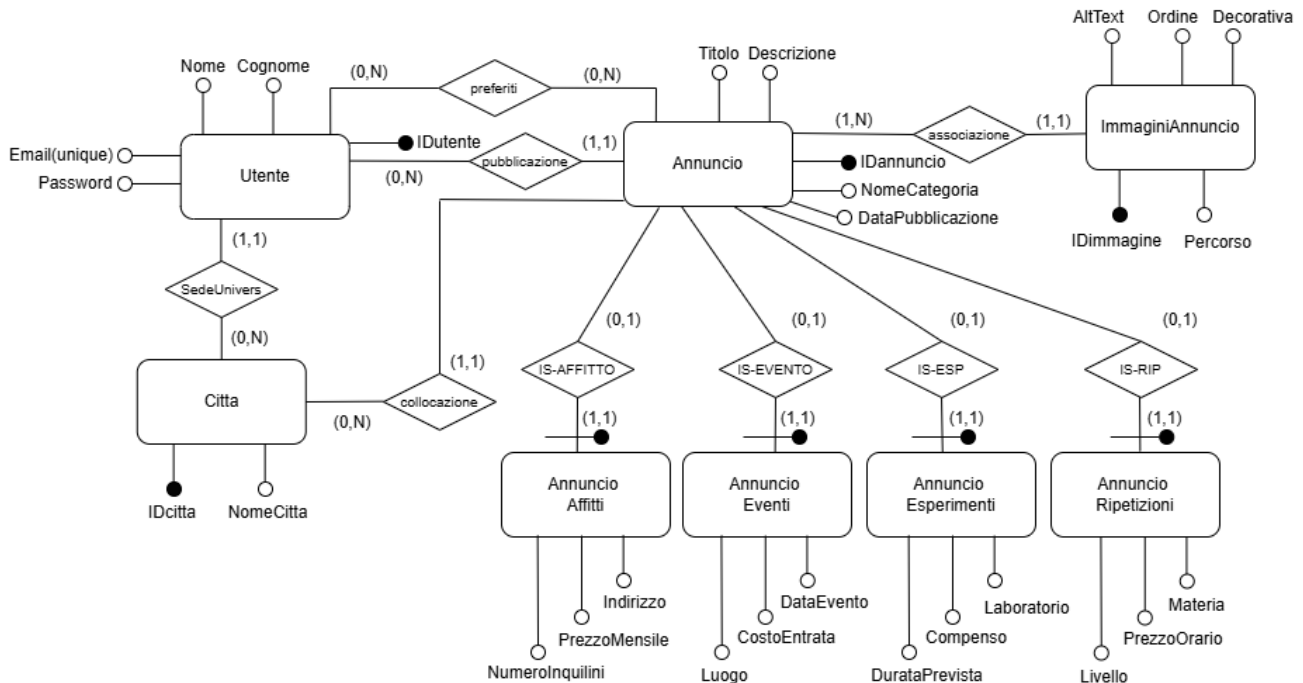


Figura 3: Schema ER ristrutturato

4.1.4 Schema relazionale

L'asterisco indica attributi opzionali.

- **Utente**(IdUtente, Nome, Cognome, Email [UNIQUE], Password, SedeUniversitaria*)
 - Utente.SedeUniversitaria $\rightarrow$ Città.Idcittà
- **Città**(Idcittà, Nomecittà [UNIQUE])
- **Annuncio**(IdAnnuncio, Titolo, Descrizione, DataPubblicazione, Autore, Categoria, Collocazione)
 - Annuncio.Autore $\rightarrow$ Utente.IdUtente
 - Annuncio.Collocazione $\rightarrow$ Città.Idcittà
- **Affitti**(IdAnnuncio, PrezzoMensileAff, IndirizzoAffitti, NumeroCoinquilini)
 - Affitti.IdAnnuncio $\rightarrow$ Annuncio.IdAnnuncio
- **Esperimenti**(IdAnnuncio, LaboratorioEsp, DurataPrevEsp, CompensoEsp)

- $\text{Esperimenti.IdAnnuncio} \rightarrow \text{Annuncio.IdAnnuncio}$
- **Eventi**(IdAnnuncio, DataEventi, LuogoEventi, CostoEntrataEventi)
 - $\text{Eventi.IdAnnuncio} \rightarrow \text{Annuncio.IdAnnuncio}$
- **Ripetizioni**(IdAnnuncio, MateriaRipetizioni, LivelloRip, PrezzoOrarioRip)
 - $\text{Ripetizioni.IdAnnuncio} \rightarrow \text{Annuncio.IdAnnuncio}$
- **ImmaginiAnnuncio**(IdImmagine, IdAnnuncio, Percorso, AltText*, Decorativa, Ordine)
 - $\text{ImmaginiAnnuncio.IdAnnuncio} \rightarrow \text{Annuncio.IdAnnuncio}$
- **Preferiti**(IdAnnuncio, IdUtente)
 - $\text{Preferiti.IdAnnuncio} \rightarrow \text{Annuncio.IdAnnuncio}$
 - $\text{Preferiti.IdUtente} \rightarrow \text{Utente.IdUtente}$

4.1.5 Normalizzazione

Lo schema progettato rispetta la terza forma normale (3NF):

- **1NF**: tutti gli attributi sono atomici; non sono presenti campi multivalore o ripetuti.
- **2NF**: ogni tabella possiede una chiave primaria semplice (IdUtente, Idcittà, IdAnnuncio). Nelle tabelle di specializzazione (Affitti, Esperimenti, Eventi, Ripetizioni) la chiave primaria è IdAnnuncio, da cui tutti gli attributi dipendono interamente.
- **3NF**: non esistono dipendenze transitive tra attributi non chiave; ogni attributo dipende direttamente e unicamente dalla chiave primaria della propria tabella.

4.1.6 Vincoli di integrità

Lo schema logico rispetta i principali vincoli di integrità:

- **Unicità**: Email dell'utente, Nomecittà (e, se previsto, NomeCategoria) sono vincolati come UNIQUE.
- **Dominio**: gli attributi obbligatori sono definiti come NOT NULL, garantendo la completezza dei dati.
- **Referenziali**:
 - **ON DELETE CASCADE** nelle tabelle di specializzazione e in Preferiti, per eliminare automaticamente i record dipendenti.
 - **SET NULL** in Annuncio per gli attributi che fanno riferimento a entità eliminabili (utente, città), così da preservare la consistenza dello storico.

- **Generalizzazione IS-A:** ogni annuncio può appartenere ad una sola specializzazione, coerente con la categoria selezionata.

4.2 HTML

Ogni documento è organizzato nelle sezioni `<head>` e `<body>`, con uso coerente di elementi semantici ed evitando tag puramente presentazionali.

4.2.1 Meta tag title, description e keywords

I meta tag sono definiti in modo coerente con il contenuto della pagina, così da garantire una corretta indicizzazione e una corrispondenza semantica tra metadati e contenuto visibile. Le **keywords** compaiono anche nel testo (headings, **strong**), evitando discrepanze e possibili penalizzazioni.

Nelle pagine dinamiche (`annuncio.html`, `modificaAnnuncio.html`) i meta tag vengono generati automaticamente sulla base dei dati dell'annuncio (titolo, categoria, città).

Il titolo è prodotto tramite la funzione `Tool::titoloSEO(string, string, string, bool)`, che impone un limite di 55 caratteri, gestisce prefissi/suffissi e accorcia il testo sull'ultima parola completa.

Anche la **description** è generata dinamicamente quando necessario, così da descrivere in modo sintetico e coerente il contenuto della pagina.

4.2.2 Meta tag viewport

È stato inserito il meta tag: `<meta name="viewport" content="width=device-width"/>` che consente ai CSS di riferirsi alla larghezza effettiva del dispositivo, evitando l'applicazione del layout desktop su schermi ad alta densità di pixel.

4.2.3 <aside> in esplora.html

Nella pagina `esplora.html` l'elemento `<aside>` contiene i filtri di ricerca, poiché rappresentano contenuti accessori rispetto al flusso principale (lista degli annunci).

4.2.4 Struttura di cardTemplate

Le card degli annunci (`cardTemplate.html`) utilizzano `<h3>` per i titoli, evitando coppie `<dt>`–`<dd>`. La gerarchia semantica è così preservata: `<h2>` identifica la sezione (es. “Ultimi inseriti”), mentre `<h3>` rappresenta ciascun annuncio.

4.2.5 Gestione delle immagini

La gestione delle immagini è progettata per ottimizzare prestazioni e accessibilità.

- Le immagini above-the-fold usano `loading="eager"`, tutte le altre `loading="lazy"`.
- Formati adottati: WebP per immagini decorative, SVG per il logo e JPG per le immagini degli annunci.

- Tutte le immagini sono inferiori a 1 MB.

Le immagini decorative sono inserite nel markup e dimensionate via CSS; il logo utilizza tecniche di image replacement per mantenere accessibile il testo alternativo mentre le immagini degli annunci hanno dimensioni fissate via HTML per rendere l'esperienza utente migliore in quando ciò garantisce un rendering più veloce.

Nel form di pubblicazione/modifica l'utente deve rispettare vincoli espliciti (max 1 MB, formato JPG), infatti è presente un controllo sulla dimensione dei file.

Si è scelto di limitare i MIME type accettati, rinunciando a conversioni automatiche o all'utilizzo del tag `<picture>` con doppia estensione `.png` per la stampa e `.jpg` per il web, per privilegiare semplicità e controllo.

La gestione dell'attributo `alt` è affidata all'utente: ogni immagine deve avere un testo alternativo oppure essere marcata come decorativa, e non è possibile lasciare entrambi i campi vuoti. Il testo alternativo è limitato a 100 caratteri per garantire sintesi e usabilità con screen reader.

4.2.6 Form e validazione

La struttura dei form utilizza `fieldset` e `legend`, anche quando destinati principalmente alle tecnologie assistive. Nessun campo è figlio diretto di `form`.

Ogni input utilizza il tipo HTML più appropriato ed è sempre associato a una `label`. Dove utile, sono abilitati `autocomplete` e `spellcheck`.

La validazione sfrutta: controlli HTML5 (`required`, `min`, `max`, `step`, vincoli di tipo); validazione JavaScript per feedback più dettagliati ed istantanei; validazione lato server per garantire correttezza e sicurezza.

4.2.7 Messaggio informativo nei redirect verso la pagina di accesso

Quando l'utente viene reindirizzato alla pagina di accesso a seguito del tentativo di accedere a una funzionalità riservata, viene mostrato automaticamente un riquadro informativo che spiega il motivo del redirect, con un messaggio contestuale alla pagina di provenienza (pubblicazione, modifica, profilo o salvataggio nei preferiti).

4.3 CSS e decisioni di design

Il foglio di stile è stato progettato privilegiando l'uso di dimensioni relative (`em`, `rem`, percentuali e unità flessibili), così da garantire un'interfaccia adattabile a dispositivi e contesti differenti, migliorando accessibilità e scalabilità.

Per assicurare uniformità grafica è stata definita una `:root` contenente variabili CSS per colori, spaziature, raggi di arrotondamento, ombre, transizioni e una scala tipografica coerente per dimensioni e pesi dei caratteri.

Il layout è realizzato principalmente tramite Flexbox, utilizzato per strutture fluide e adattive, mentre CSS Grid è impiegato solo nei casi in cui risulta più appropriato (ad esempio per elenchi di card o sezioni a griglia).

La responsività è gestita tramite la separazione tra `style.css` e `mini.css`, con un breakpoint principale a 800px per l'adattamento ai dispositivi mobili. Sono presenti due ulteriori media query, tra cui una dedicata alla navigazione con menù hamburger, attivata sotto i 1000px per migliorare l'usabilità su tablet e schermi intermedi.

È stato inoltre previsto un foglio di stile dedicato alla stampa, che mostra esclusivamente le informazioni essenziali, rimuovendo elementi decorativi e interattivi per ridurre il consumo di inchiostro e mantenere una struttura coerente del contenuto.

Particolare attenzione è stata riservata all'accessibilità visiva e all'interazione da tastiera, con la gestione esplicita degli stati di focus visibile, l'uso coerente dei contrasti e l'eliminazione di animazioni superflue nei contesti in cui potrebbero risultare problematiche, come nelle versioni mobile o di stampa.

4.4 PHP

5 Accessibilità

5.1 Strumenti di verifica

Per garantire il rispetto dei requisiti WCAG 2.2, è stata compilata la checklist fornita in Moodle.

Sono stati utilizzati diversi strumenti di verifica:

- **NVDA**: per testare la gerarchia dei titoli, dei link e dei punti di riferimento, la navigazione tramite landmark e l'annuncio dei messaggi di errore nei form, sia con JavaScript attivo sia disattivato.

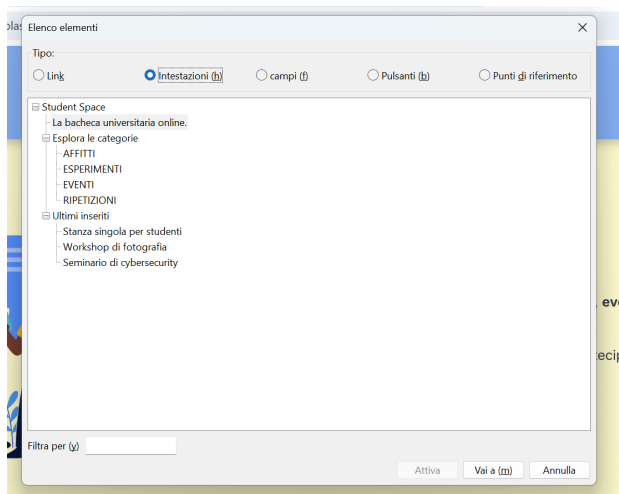


Figura 4: Gerarchia titoli (index.html)

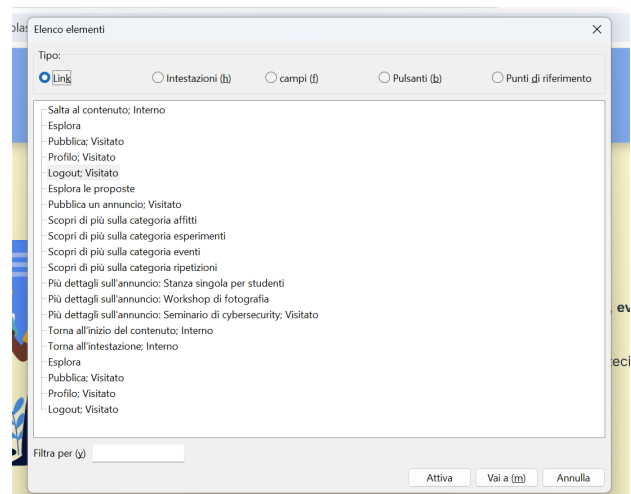


Figura 5: Lista link (index.html)

- **Contrast Checker** (<https://contrastchecker.com/>): per verificare il rispetto del rapporto di contrasto $\geq 4.5:1$.

- **Lighthouse, Silktide e strumenti di ispezione (F12):** per valutazioni automatiche e test manuali su navigazione da tastiera, accessibilità dei link, dimensioni degli elementi cliccabili e altri criteri WCAG.

5.2 Tipografia e leggibilità

Per garantire una lettura chiara e accessibile sono stati scelti due font distinti:

- **Web:** *Noto Sans*, importato tramite Google Fonts.
- **Stampa:** *Times New Roman*.

La scelta del font web è stata guidata da verifiche sulla leggibilità di caratteri critici (*qp*, *db*, *00*, *iIl1*, *rn*, *m*), sull'altezza della *x* e sull'apertura delle lettere *c* e *a*. Inizialmente era stato considerato *Inter*, poi sostituito da *Noto Sans* per una migliore distinzione di glifi come *ca* e *iIl1*.

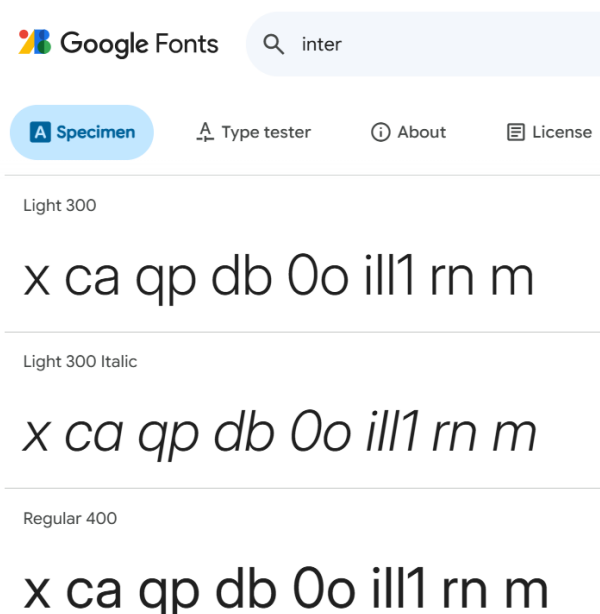


Figura 6: Font: Inter

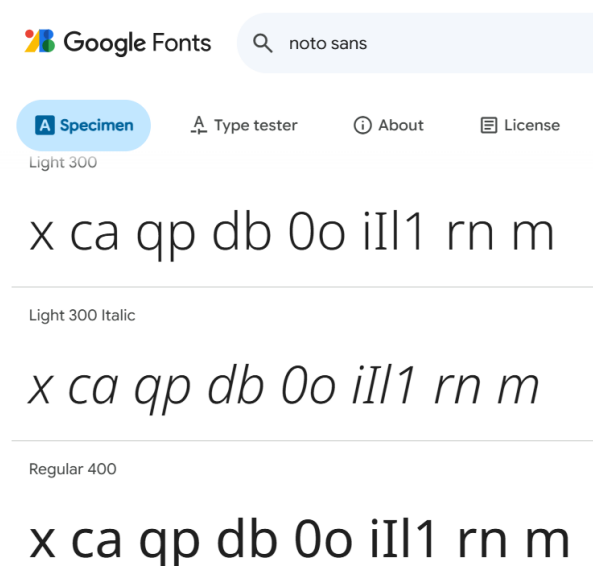


Figura 7: Font: Noto Sans

Leggibilità dei prezzi La funzione `PHP Tool::formatPrezzo()` formatta i valori numerici mostrando i decimali solo quando necessario (es. 20€ invece di 20.00€), migliorando la leggibilità.

5.3 Colori, contrasti e accessibilità mobile

Sono stati adottati diversi accorgimenti per garantire una corretta percezione dei contenuti e la riconoscibilità degli elementi interattivi:

- **Link e CTA:**
 - I link testuali sono sempre sottolineati.

- I link visitati/non visitati nella navbar differiscono per colore (contrasto $2.72 \approx 3$).
- I pulsanti CTA variano colore tra stato visitato e non visitato.
- **Contrasto complessivo:** tutti i contrasti rispettano il rapporto minimo WCAG ($\geq 4.5:1$), verificato tramite Contrast Checker.
- **Accessibilità touch su mobile:**
 - Gli elementi cliccabili rispettano dimensioni minime di circa $44\text{px} \times 44\text{px}$.
 - Il pannello laterale dei filtri utilizza un overlay e una gestione del focus accessibile (`toggleFiltriAccessibile()`).
 - I pulsanti dei caroselli hanno opacità testata e leggibile (`rgba(22, 40, 110, 0.65)`).

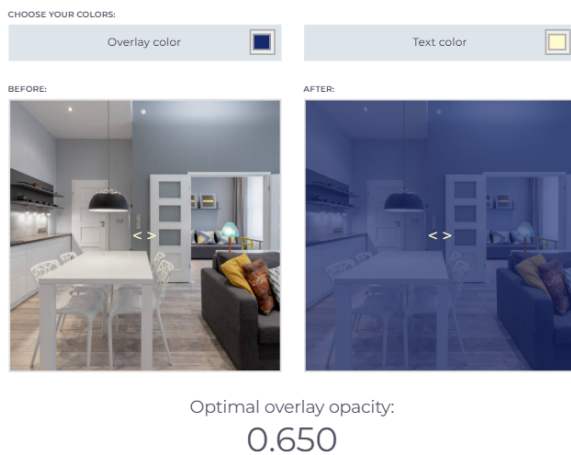


Figura 8: Opacità su immagine prova 1

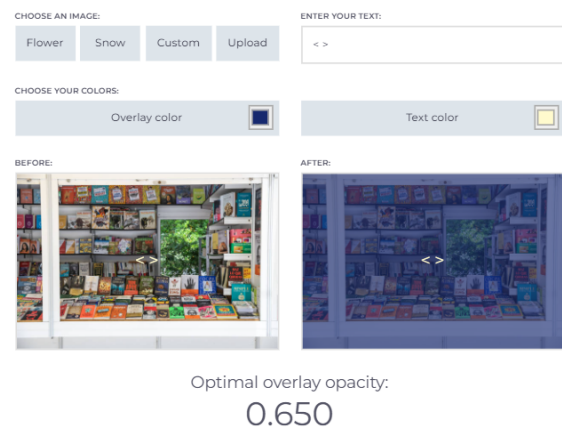


Figura 9: Opacità su immagine prova 2

6 Validazione, sicurezza e gestione degli errori

6.0.1 Errori lato client (JavaScript)

La validazione lato client è attiva su tutti i form con attributo `data-validate`, combinando controlli in tempo reale e al submit.

- **Eventi gestiti:** `blur`, `input`, `submit`.
- **Messaggi di errore:** generati come `<ul><li>` con `role="alert"`; gli input ricevono `aria-invalid` e `aria-describedby`.
- **Focus:** portato sul primo campo non valido solo alla prima comparsa dell'errore.
- **Campi gestiti:** testo, email, password, conferma password, checkbox, città con `datalist`.
- **Funzioni principali:** `validateField()`, `renderError()`, `removeError()`.

6.0.2 Errori lato server (PHP)

La validazione lato server garantisce feedback anche senza JavaScript.

- **Tipi di errori:** campi generici, immagini (dimensione, formato, alt), login.
- **Messaggi:** generati come liste con `role="alert"`, uniformi a quelli client-side.
- **Attributi ARIA:** `aria-invalid` e `aria-describedby` applicati dinamicamente.
- **Focus:** portato sul primo campo non valido; per le immagini, sul primo input `file`.
- **Errori immagini:** controllo dimensione (1MB), formati (JPG, PNG, WEBP), obbligatorietà del testo alternativo.
- **Login:** gestione errori credenziali, HTML non consentito, messaggi contestuali tramite `redirectMessage`.
- **Uniformità:** stesso markup, stessi stili e stessa semantica dei messaggi client-side.

7 Configurazione .htaccess

8 Test

Per verificare la qualità complessiva del sito, sono stati effettuati test approfonditi su più livelli: accessibilità, usabilità, robustezza del codice e comportamento multiplatforma.

8.1 Testing multiplatforma con e senza JavaScript

Il sito è stato testato su diversi browser (Chrome, Firefox, Edge) e su dispositivi differenti (desktop e smartphone). Sono state eseguite prove sia con JavaScript attivo sia con JavaScript disattivato, per verificare che i contenuti principali rimanessero accessibili e che la navigazione non risultasse compromessa. In assenza di JavaScript, sono stati controllati in particolare la leggibilità dei contenuti, la presenza di percorsi alternativi e la corretta visualizzazione delle sezioni informative.

8.2 Ordine di tabulazione e navigazione da tastiera

È stato verificato l'ordine di tabulazione da desktop e da mobile, con particolare attenzione all'*aside* dei filtri, che utilizza una tendina a scomparsa con overlay. Sono stati controllati:

- la corretta gestione del focus quando il pannello dei filtri viene aperto o chiuso;
- l'impossibilità di interagire con gli elementi oscurati dall'overlay;
- la presenza di un percorso di tabulazione lineare e prevedibile;
- la possibilità di raggiungere e attivare tutte le azioni tramite tastiera.

La navigazione da tastiera è stata testata sull'intero sito, verificando che pulsanti, link, menù e componenti interattivi fossero correttamente etichettati e attivabili.

8.3 Analisi delle prestazioni e SEO

È stato utilizzato *Lighthouse* per analizzare prestazioni, accessibilità, SEO e buone pratiche. I test hanno permesso di individuare eventuali elementi migliorabili, come ottimizzazioni delle immagini e riduzione del tempo di caricamento delle pagine.

8.4 Validazione con strumenti automatici

Sono stati utilizzati diversi strumenti per verificare la qualità del codice e l'accessibilità:

- **WAVE WebAIM:** utilizzato anche con gli stili disattivati per controllare il flusso logico dell'HTML e la corretta struttura semantica;
- **Silktide:** per un'analisi aggiuntiva di accessibilità e best practice;
- **Total Validator:** utile per validare le diverse casistiche generate dinamicamente dal PHP, verificando che ogni variante di pagina rispettasse gli standard HTML5;
- **W3B1Y:** per ulteriori controlli di accessibilità e conformità alle WCAG.

L'uso combinato di questi strumenti ha permesso di individuare e correggere errori strutturali, etichette mancanti, ruoli ARIA non necessari e possibili problemi di navigazione assistita.

8.5 Studio comportamento utenti

Per garantire che il comportamento del sito fosse chiaro, prevedibile e accessibile, abbiamo svolto una serie di test con utenti esterni al gruppo di sviluppo. Essendo noi i creatori del sistema, avremmo potuto non accorgerci di elementi poco intuitivi; per questo abbiamo coinvolto persone con profili diversi: studenti di varie facoltà, ex studenti, genitori e una persona con mobilità limitata che utilizza una sola mano. Questo ci ha permesso di osservare l'interazione da prospettive differenti e di verificare che l'esperienza d'uso rimanesse comprensibile e coerente per utenti con esigenze e competenze eterogenee.

9 Strumenti utilizzati

- **Canva** https://www.canva.com/it_it/
Piattaforma di progettazione grafica per creare contenuti visuali per presentazioni, siti web e prodotti simili.
- **Discord** <https://discord.com/>
Piattaforma di messaggistica istantanea e distribuzione digitale, utilizzata per la comunicazione del gruppo. Supporta chiamate vocali, videochiamate, messaggi di testo e condivisione file.

- **Git** <https://git-scm.com/>
Applicazione software per il controllo di versione distribuito, usata per il versionamento del codice e della documentazione.
- **Github** <https://github.com/>
Servizio di hosting per progetti software che implementa Git, utilizzato per la collaborazione e la condivisione del codice.
- **LaTeX** <https://www.latex-project.org/get/>
Linguaggio di marcatura del testo, usato per la preparazione della relazione accademica.
- **Silktide** <https://silktide.com/>
Estensione che facilita i test sull'accessibilità.
- **W3C Validator** <http://validator.w3.org/>
Strumento per validare il markup HTML/XHTML.
- **WhatsApp** <https://www.whatsapp.com/>
Piattaforma di messaggistica istantanea usata per la comunicazione rapida tra i membri del gruppo.
- **WebAIM** <https://webaim.org/resources/contrastchecker/>
Strumento online per verificare il contrasto dei colori e migliorare l'accessibilità.

9.1 Docker

Per evitare di incorrere in problematiche dovute alle differenze tra i vari sistemi utilizzati dai membri del team (dipendenze, versioni differenti del SW, OS ecc.), si è deciso di fare largo uso di Docker, in modo da poter lavorare tutti su una base comune. Per tale scopo è stata creata (e poi raffinata nel tempo) un'immagine Docker che mettesse a disposizione:

- un database in MariaDB, versione 10.6.7
- un server PHP Apache, versione 8.2
- un'istanza di phpMyAdmin, per l'accesso e amministrazione del database

Tale configurazione è stata molto utile per consentire, durante lo sviluppo, di visualizzare le modifiche senza il timore che potessero avere effetti diversi, su altri sistemi, da quelli desiderati. Ha permesso inoltre di velocizzare le attività, in quanto tutti i servizi necessari venivano richiamati automaticamente da un apposito docker compose.

10 Guida all'installazione del progetto